

Relatório de Análise de Algoritmos de Ordenação

Este trabalho tem como objetivo analisar comparativamente o desempenho de três algoritmos clássicos de ordenação: Bubble Sort, Insertion Sort e Quick Sort.

A análise foi realizada através da medição do tempo de execução de cada algoritmo aplicado a diferentes conjuntos de dados, variando tanto em tamanho (100, 1.000 e 10.000 elementos) em ordens diferentes (aleatório, crescente e decrescente).

Metodologia:

- Bubble Sort: Implementação recursiva com otimização através da flag swapped para detecção de ordenação completa
- Insertion Sort: Implementação iterativa com inserção direta na posição correta
- Quick Sort: Implementação recursiva com pivô fixo (último elemento)

Resultados:

- Foram obtidos os seguintes resultados para o relatório:

ALGORITMOS DE ORDENAÇÃO				
Tipo de Dados	Tamanho	Bubble Sort (ms)	Insertion Sort (ms)	Quick Sort (ms)
Aleatorio	100	1,5024	0,1881	0,1061
Aleatorio	1000	9,4615	6,2147	0,8071
Aleatorio	10000	439,5423	44,7627	3,4806
Crescente	100	0,0071	0,0082	0,1203
Crescente	1000	0,0120	0,0162	1,6594
Crescente	10000	0,0332	0,1039	140,2526
Decrescente	100	0,0976	0,0121	0,0505
Decrescente	1000	13,9142	0,8231	1,8342
Decrescente	10000	192,4934	70,5110	139,6612

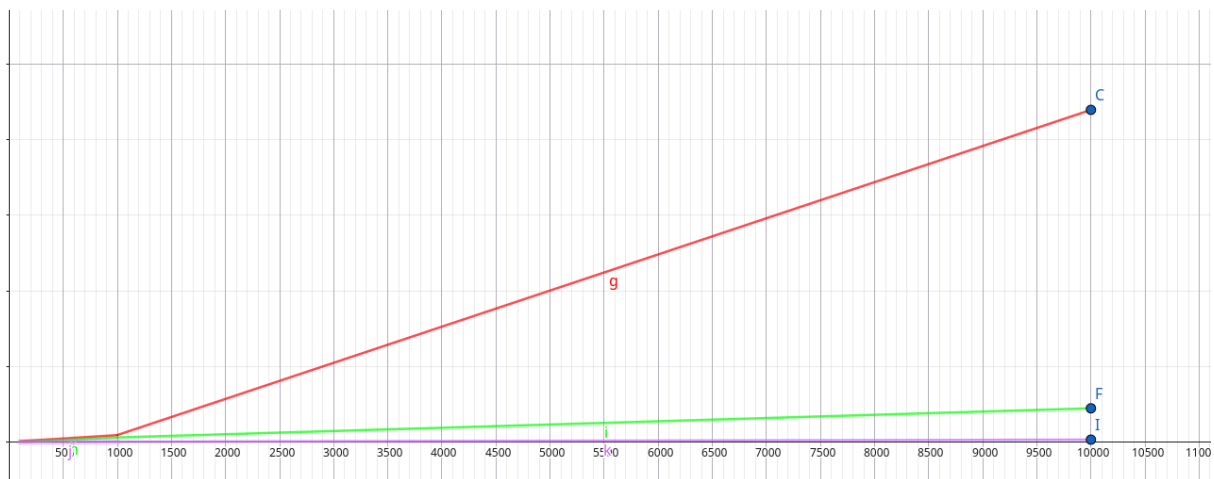
Análise dos Resultados:

- Para maior análise, vamos separar os tipos de dados e colocar em um gráfico.

Orientações:

- Linha vermelha: Algoritmo Bubble Sort
- Linha verde: Algoritmo Insertion Sort
- Linha roxa: Algoritmo Quick Sort
- *obs: esse padrão será repetido

Primeiramente, para os dados formados de maneira aleatória:

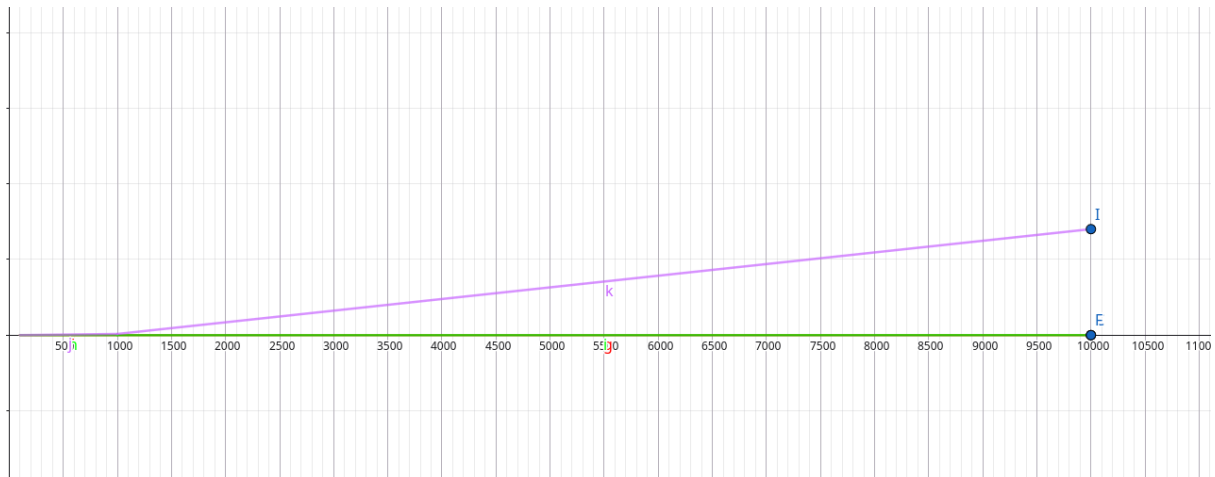


O Quick Sort apresentou o melhor desempenho, se destacando notoriamente com aproximadamente 3 ms para 10.000 dados, sendo aproximadamente 100 vezes mais rápido que o Bubble Sort e aproximadamente 10 vezes mais rápido que o Insertion Sort.

O Insertion Sort foi mais eficiente que o Bubble Sort devido a menos movimentações desnecessárias.

O Bubble Sort apresentou o pior desempenho devido ao seu crescimento de maneira quadrática, onde ao aumentar o tamanho 10 vezes, seu tempo cresceu aproximadamente 100 vezes.

Por outro lado, para o caso em que os dados já estão crescentes:



Como pode-se notar, o Bubble Sort obteve o melhor desempenho graças à flag swapped, onde após uma única passada sem nenhuma troca, o algoritmo detecta que já está ordenado e para.

O Insertion Sort foi um pouco mais lento que o Bubble Sort devido ao fato de percorrer todo o array, fazendo verificações, enquanto o Bubble Sort para na primeira passada.

Já o Quick Sort apresentou um desempenho inferior para 10.000 elementos devido ao pivô mal escolhido. Como sempre é escolhido o último elemento como pivô e os dados estão em ordem crescente, o pivô é sempre o maior valor. Isso gera divisões desbalanceadas, onde todos os elementos ficam à esquerda do pivô e nenhum à direita.

Para o caso de quando os dados estão de maneira decrescente, o resultado é ainda diferente:



O Insertion Sort apresentou o melhor desempenho, sendo mais eficiente que algoritmos que fazem trocas individuais. Em dados decrescentes, cada novo elemento precisa ir para o início, mas o Insertion faz isso deslocando todos de uma vez.

O Quick Sort também sofreu pelo mesmo problema de pivô desbalanceado: como o pivô é sempre o último elemento e os dados estão em ordem decrescente, o pivô é sempre o menor valor. Isso faz com que todos os elementos fiquem à direita do pivô e nenhum à esquerda, gerando divisões desbalanceadas.

O Bubble Sort foi o mais lento pois precisa fazer múltiplas passadas, cada uma realizando várias trocas individuais para mover os elementos maiores para o final. Diferente dos dados crescentes onde a flag swapped permite parar imediatamente, em dados decrescentes não há essa vantagem.

Também pode-se perceber um fenômeno: para 100 dados, o Bubble Sort obteve resultado mais otimizado do que o Quick Sort, mas então para 1000 e 10000 dados, obteve resultados mais lentos.

Aprendizados

Não há um único algoritmo certo ou errado para fazer uma ordenação, mas existem algoritmos mais adequados para cada cenário: o Bubble Sort se saiu melhor quando os resultados já estão ordenados, o Insertion Sort possui resultados melhores para inverter a ordenação dos dados e o Quick Sort quando os dados estão de fato escritos de maneira aleatória.

Como há casos na programação em que são necessários a ordenação de dados aleatórios, a inversão da ordenação de um conjunto de dados e também há casos em que os dados estão quase ordenados, ou seja, precisam de pouca ordenação, pode-se tirar proveito de todos os algoritmos em diferentes ocasiões.